HF Model Finder: Constraint Verified Retrieval-Augmented Generation for Finding Hugging Face Models

Report submitted in partial fulfilment

of

B.Tech

in

Computer Engineering

by

Harsh Bhandarkar (B014), **Armaan Chopra** (B028),
Jay Gandhi (B124), **Aditya Rane** (B129),
Yash Parab (B179)

Under the supervision of

Dr. Archana Nanade

(Professor, Computer Engineering dept., MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

(2026-27)

ABSTRACT

Choosing a ready-made AI model on Hugging Face is harder than it looks. A developer might ask for "a Marathi named-entity model under 500 MB that I can use commercially". Most of this request is made of hard rules: the task, the language, the size and the licence. A model either meets them or it does not. Ordinary search tools and ordinary RAG (retrieval-augmented generation) systems do not check these rules properly and suggest whatever sounds relevant. In our tests, plain RAG broke at least one rule in half of its suggestions.

This report presents HF Model Finder. A language model only reads the user's request and later writes a short explanation. All the pass or fail checks are done by ordinary code that reads the model's metadata, so a model card can never talk its way into a pass. When the metadata is missing, the tool says "unknown" instead of guessing. It searches 15,991 models across eleven tasks, with extra focus on Indian languages, and has a simple web page built with Streamlit.

We compared it with plain RAG, keyword search and Hugging Face's own filters on 100 test questions written before we tuned anything. On 75 normal questions it never suggested a model that breaks a rule (plain RAG: 50.4%; Hugging Face filters: 25.9% to 32.8%), and 83.3% of its suggestions were fully checked (plain RAG: 29.1%). It asked a follow-up question on all 15 vague requests. A relevance test graded by AI judges, not people, also gave it the best score. The main limits are that about 27% to 30% of models have no known size, that grading was done by AI judges, and that the six newer tasks were only spot-checked.

Note: the software was developed with the assistance of an AI coding assistant (Claude Code); see Section 3.8. Relevance and faithfulness grades were produced by AI judges, not humans.

CONTENTS

Chapter 1 Introduction.....1
Chapter 2 Literature Survey.....2
Chapter 3 Methodology and Implementation.....3
Chapter 4 Screenshots and Results.....6
Chapter 5 Source Code.....8
Chapter 6 Conclusion and Future Work.....9
References.....9
Appendix A: UML Activity Diagrams.....10

Chapter 1 Introduction

A model website is only useful if you can find the right model on it. Hugging Face hosts more than a million ready-made AI models. Suppose a developer wants "a small Marathi named-entity model that I can use in a commercial product". Today they have to turn this into filters for task, language, licence and size, and then open model pages one by one to check. We built HF Model Finder to do this for them, and to make sure the answer can be trusted on size, licence, language and task.

1.1 Background

Retrieval-augmented generation (RAG) means looking up relevant documents first and then giving them to a language model, so that it answers from real text instead of memory [1]; a survey describes the many versions of this idea [2]. Searching works in two common ways: by meaning, using "embeddings" that turn text into numbers [3], and by matching words, as in the classic BM25 method [4]. A second model can then re-order the results to put the best first [5]. Embeddings that work across many languages, including Indian ones, also exist [6]. Language models have also been used to pick Hugging Face models for a task [7]. The problem is the data itself: studies show that model documentation is often incomplete [8] and that licences are often missing or wrong on hosting sites [9].

1.2 Motivation and scope

A request like "a Marathi NER model under 500 MB, usable commercially, that works well on noisy social-media text" has one soft wish (works well on noisy text) and several hard rules (task, language, size, commercial use). Keyword search cannot understand meaning. The Hugging Face licence filter only matches a licence name and does not know what it allows. Plain RAG suggests models whose pages *sound* right: in our tests it suggested a model that breaks a rule for 50.4% of its suggestions. Data is also often missing: about 27% of the models in our first collection have no clear size.

We cover a snapshot of 15,991 models in eleven tasks: speech recognition, translation, text classification, named-entity recognition, sentence similarity, image classification, object detection, image segmentation, text generation, summarization and question answering. The tool does not run models, give legal advice or rank models by benchmark scores.

1.3 Problem statement

Build and test a tool that suggests Hugging Face models and checks every suggestion, in code, against each hard rule the user gave (task, language, size, commercial use, modification rights); that says "unknown" when the data is missing; that asks a question when the request is unclear; and that does better on these rules than plain RAG, keyword search and the Hugging Face filters, with a focus on Indian-language models.

1.4 What we contributed

1. **Code does the checking.** The language model only reads the request and writes explanations. Pass, fail or unknown is decided by ordinary code from the model's data.
2. **"Unknown" is a real answer.** Missing data is shown, not guessed, and a banner appears when nothing could be fully checked.
3. **Licences are turned into rights** (can I use it commercially? can I change it?) and size is calculated by one fixed rule.
4. **Check many, show few.** We check 100 models and show the best 5.
5. **Easy for beginners:** follow-up questions, a plain-English summary of what we understood, a starter code snippet and fine-tuning advice.

6. **A fair test** on 100 questions written before tuning, with its limits stated.

Chapter 2 Literature Survey

2.1 What others have done

RAG. Lewis *et al.* [1] joined a text generator to a search system over Wikipedia, so the model could look things up. It gave better and more factual answers and the knowledge could be updated by changing the search index. But the retrieved items are plain text, so nothing forces them to satisfy a rule such as "under 500 MB". Gao *et al.* [2] review the whole family of RAG methods and group them into simple, advanced and modular designs. Our system is a modular RAG with one extra step: checking.

Searching and ranking. Karpukhin *et al.* [3] showed that searching by meaning (dense retrieval) beats BM25 by 9 to 19 points on question answering. Robertson and Zaragoza [4] describe BM25, the standard word-matching method, which we use as a baseline. Nogueira and Cho [5] used a BERT model to re-order search results and improved a standard benchmark by 27%; this is why we re-rank after searching. Wang *et al.* [6] built multilingual embeddings for about a hundred languages, and we use their model so that Hindi and Marathi requests work.

Choosing models with a language model. HuggingGPT [7] lets a language model read descriptions of Hugging Face models and pick some for a job. It decides from text only, so it never checks size, licence or language.

Data quality. Jiang *et al.* [8] interviewed practitioners who reuse models from Hugging Face and found three problems: missing information, claims that do not match real performance, and risks. Longpre *et al.* [9] checked more than 1,800 datasets and found licences missing in over 70% of cases and wrong in over 50% on popular hosting sites. This is why our tool says "unknown" and why it says that rights come from the *declared* licence.

Safety and testing. Greshake *et al.* [10] showed that text fetched from the web can contain hidden instructions that take over a language model. Model cards can be written by anyone, so our checks never depend on what the language model says. Zheng *et al.* [11] found that strong language models can grade other models' answers and agree with people more than 80% of the time, but have biases; we use this idea for grading and say so openly.

2.2 Comparison

Table 2.1. Related work compared with what this project needs.

Work	What it gives	Checks hard rules?	Gap for this problem
RAG [1], [2]	Search plus generation over text	no	Only checks relevance
Dense search, BM25, re-ranking [3]–[5]	Finding and ordering results	no	No idea of rules
HuggingGPT [7]	A language model picks Hub models	no	Picks from text, no checks
Reuse study, licence audit [8], [9]	Shows data is missing or wrong	n/a	No tool
This work	Code checks every rule; "unknown" allowed	**yes**	Tested on 100 questions

2.3 What is missing

Four gaps remain. (1) No method guarantees that suggested models follow the user's hard rules. (2) Missing data is not handled: the system needs to be able to say "the data does not say". (3) Licences are shown but not turned into rights such as "commercial use: yes or no". (4) Indian-language models, which are many but rarely downloaded, are hard to find. Our project fills these four gaps.

Chapter 3 Methodology and Implementation

3.1 What the system must do

Table 3.1. The main requirements and how each was checked.

ID	The system must	Checked by
R1	show, in plain words, how it understood the request, and let the user edit it	Screenshots
R2	turn the request into rules (task, language, size, licence rights) and validate them	Unit tests; 29 test requests
R3	ask a follow-up question if the task or language pair is missing (at most two)	15 vague queries
R4	decide pass, fail or unknown for each rule, in code, from metadata only	Unit tests; violation rate
R5	never suggest a model that fails a rule the user gave	75 normal queries
R6	show "unknown" when data is missing, never guess	Unknown rate
R7	say clearly when no suggested model is fully checked	12-query test
R8	keep model-card text from changing any verdict	Poisoned-card test

3.2 Component diagram

Fig. 3.1 is a UML component diagram of the system. The **offline** part runs once: it downloads model information from Hugging Face, cleans it, keeps the facts in a snapshot, and stores the page text as vectors in the Pinecone index. The **online** part runs for every question: the web page sends the request to the query pipeline, which uses the language model, searches the index, and asks the verifier to check the rules using the snapshot. The two parts meet only at the data stores.

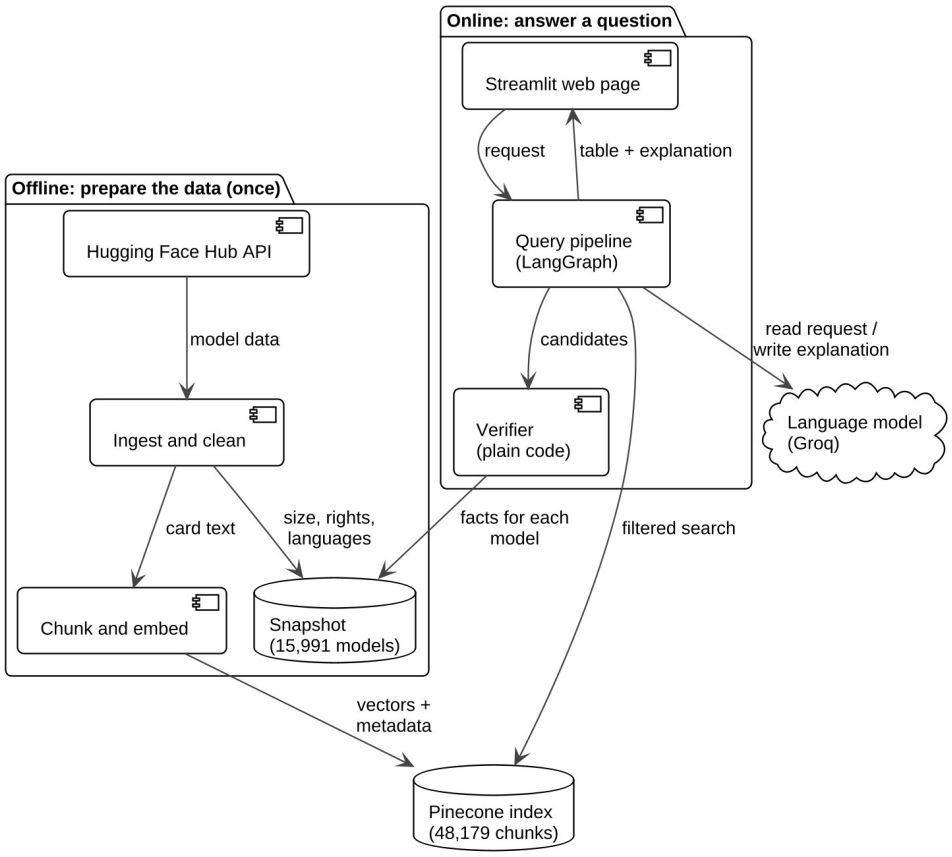


Fig. 3.1. UML component diagram of HF Model Finder: the offline data preparation and the online question answering.

3.3 Hardware and software

No special hardware is needed. We worked in a cloud computer with 4 CPUs and 16 GB of memory, and no GPU. Embeddings and re-ranking come from Pinecone's hosted models (`multilingual-e5-large` and `bge-reranker-v2-m3`), and the index lives in Pinecone. The language model is `gpt-oss-20b` on Groq's free plan. The code is Python, the question pipeline uses LangGraph, the web page uses Streamlit, and 335 automatic tests check the code. One settings file holds all numbers that affect results. Pinecone's free re-ranking limit (500 requests a month) ran out on 25 October 2026, so new searches after that are not re-ranked; the page says so and the pass or fail checks are not affected.

3.4 Preparing the data

We collected 15,991 models from eleven tasks: for each task the 1,000 most downloaded models, plus all models tagged with ten Indian languages for the language tasks. Then we turned the raw information into facts that code can check:

- **Size** is the exact total of the main weight files. If we cannot tell, the size is *unknown* (29.5% of models).
- **Rights.** A table of 54 licences says whether each allows commercial use and modification: yes, no, conditional (extra terms apply) or unknown. A "conditional" licence is never counted as a clean pass.
- **Languages** are cleaned to standard codes. A few well-known multilingual models are added by hand with a source.
- **Translation direction** is known only for models named `opus-mt-<from>-<to>` (380 models); for all others it is unknown.

Model pages are cut into chunks of at most 450 tokens (roughly 300 words; at most four per model), turned into embeddings and stored in the index (48,179 chunks). A checksummed snapshot keeps the data fixed so results can be repeated.

3.5 Answering a question

Fig. 3.2 shows the order of messages for one question, and Appendix A (Fig. A.1) shows the same steps as an activity diagram. (1) **Extract:** requests for things we do not cover (image generation, OCR, speech synthesis) are refused. Otherwise the language model writes the rules as JSON, code validates them, and a small rule-based backstop adds anything obvious the model missed, such as "under 500 MB". (2) **Clarify:** if the task is missing we ask, with clickable choices. (3) **Search:** we search with the user's own words, and the filter keeps models whose data is unknown. (4) **Re-rank:** a second model orders the results. (5) **Verify:** code checks the first 100 models and keeps the best 5. (6) **Explain:** the language model writes a short explanation from the model card text, plus the page adds a code snippet and fine-tuning advice. We check 100 but show 5 because search by meaning cannot see size or licence; this raised fully checked answers from 72.3% to 83.3%.

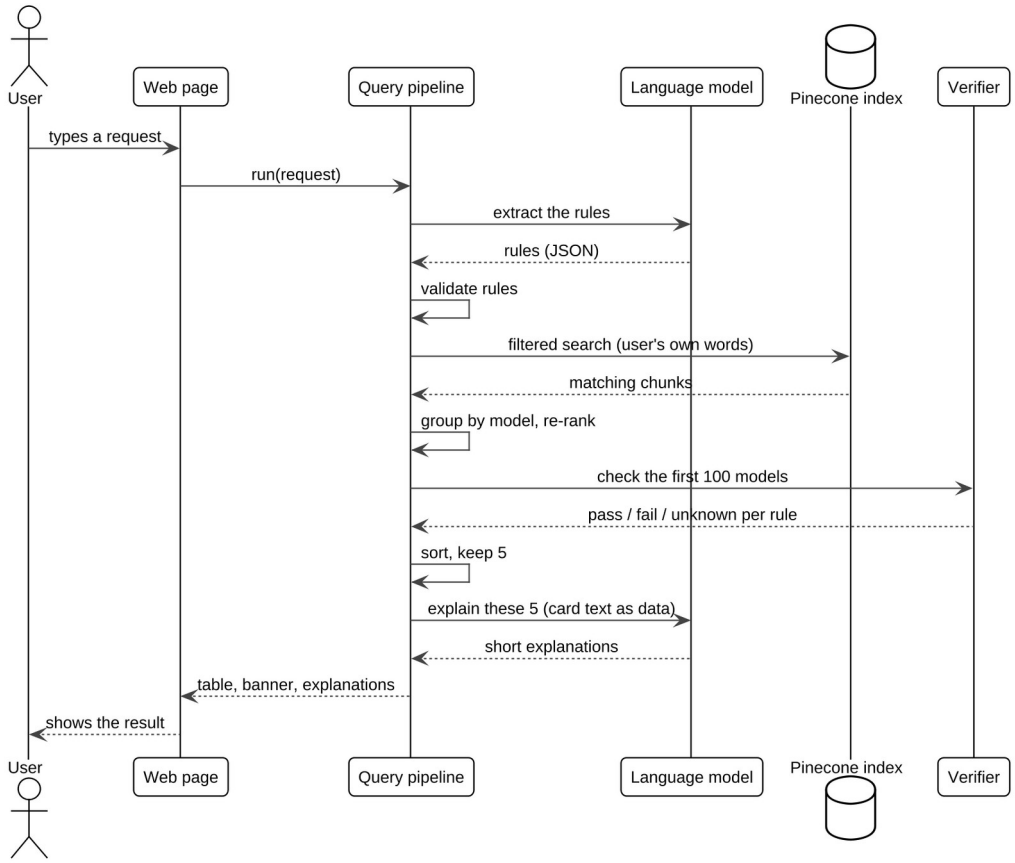


Fig. 3.2. UML sequence diagram of one question.

3.6 The checking rule

Algorithm 3.1 (and the activity diagram in Appendix A, Fig. A.2) is the only place where pass, fail or unknown is decided. A rule can fail only when the data is known and breaks the request, so a shown model can never break a rule the user gave, whatever the search or the language model does.

Algorithm 3.1. How one model is checked

```
task:       UNKNOWN if no task in data; PASS if equal; else FAIL
languages:  UNKNOWN if list empty/unknown; PASS if all requested present;
            else UNKNOWN if model is multilingual; else FAIL
direction:  PASS if the name states it; else UNKNOWN   (never FAIL)
size:       UNKNOWN if size unknown; PASS if size <= limit; else FAIL
commercial, modifiable:  PASS if yes; FAIL if no; else UNKNOWN
OVERALL = FAIL if any FAIL; else UNKNOWN if any UNKNOWN; else PASS
Sort the 100 by (OVERALL, number of UNKNOWN), drop FAIL, show the first 5.
```

3.7 Safety and testing

Anyone can write a model card, so card text could contain tricks such as "ignore your instructions" [10]. Our main protection is that checks never read model output or card text. In addition, card text is wrapped in tags and treated as data, and the language model is told not to state licence, size or language facts. We tested this with three hostile cards: the tables still came from metadata only.

To test the tool we wrote 100 questions before tuning: 75 normal, 15 vague and 10 meant to be impossible. Five systems answered them: ours, plain RAG (same index, no filters, no checks), BM25 keyword search with the same filters, and Hugging Face's filters in two forms. One shared set of rules judged all of them. Relevance was graded 0 to 2 on 686 blind (request, model) pairs, and explanations were checked claim by claim. **These grades were given by AI judges, not people.**

3.8 How the project was built

Five students worked on this project between 16 and 27 October 2026, with an AI coding assistant (Claude Code) writing and running much of the code and tests under our direction. We made the design decisions and wrote them down as we went. We also fixed the testing rules in advance: the questions were committed first, every question used to fix a bug was listed, and nothing was changed after the results were frozen for grading.

Chapter 4 Screenshots and Results

4.1 What the tool looks like

Fig. 4.1 shows the request "Small sentiment model for Hindi I can use in a commercial app" with the comparison switched on. The line "We understood" shows that "small" was read as 500 MB. On the left is our system: every row passes. On the right is plain RAG on the same data: some of its suggestions have a red cross in the size row. For vague requests such as "I want a Hindi model" the tool asks what the model should do instead of guessing. Fig. 4.2 (a) shows an honest "unknown": for "Hindi to Marathi translation under 1 MB" nothing can be fully checked, so a banner says so. Under each model, "Try it" gives a starter code snippet and "Build on it" shows whether the model can be changed, its parent model, and how much GPU memory fine-tuning needs (b).

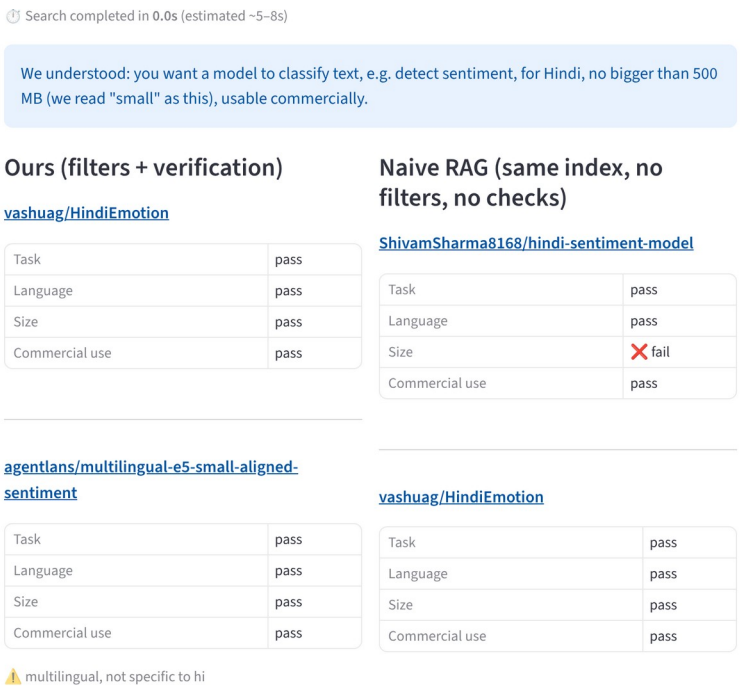
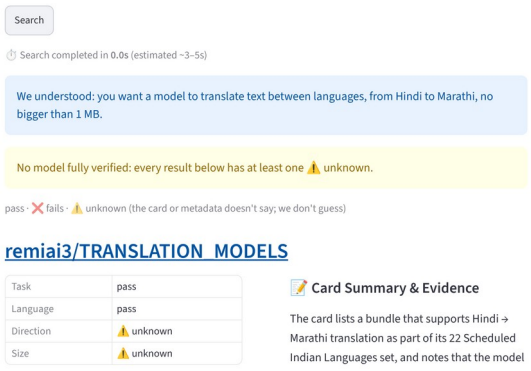
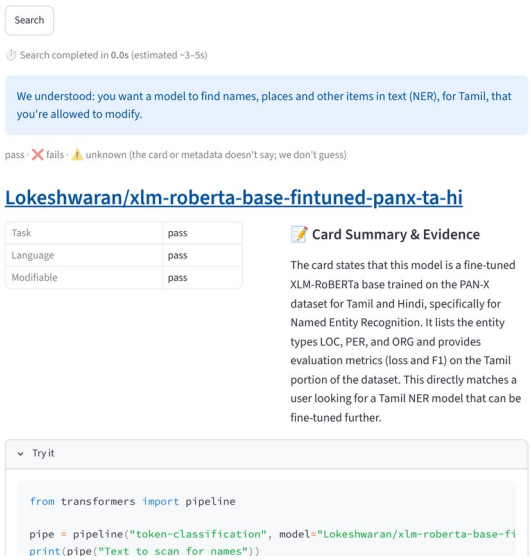


Fig. 4.1. Our results (left) against plain RAG (right) for a small Hindi sentiment model that can be used commercially.



(a) Banner when nothing is fully checked



(b) Try it and Build on it panels

Fig. 4.2. Unknowns are shown, never guessed.

4.2 Results against the other systems

Table 4.1 shows the main result on the 75 normal questions. Every system's top five models were judged by the same rules.

Table 4.1. Main results on 75 normal questions (top five per question).

System	Answered %	Models	Rule broken %	Unknown %	Fully checked %
Ours	97.3	365	**0.0**	16.7	**83.3**
Plain RAG	100.0	375	50.4	20.5	29.1
Keyword (BM25) + filters	100.0	368	0.0	62.0	38.0
Hugging Face filters, basic	100.0	375	32.8	13.6	53.6
Hugging Face filters, licence	100.0	375	25.9	14.7	59.5

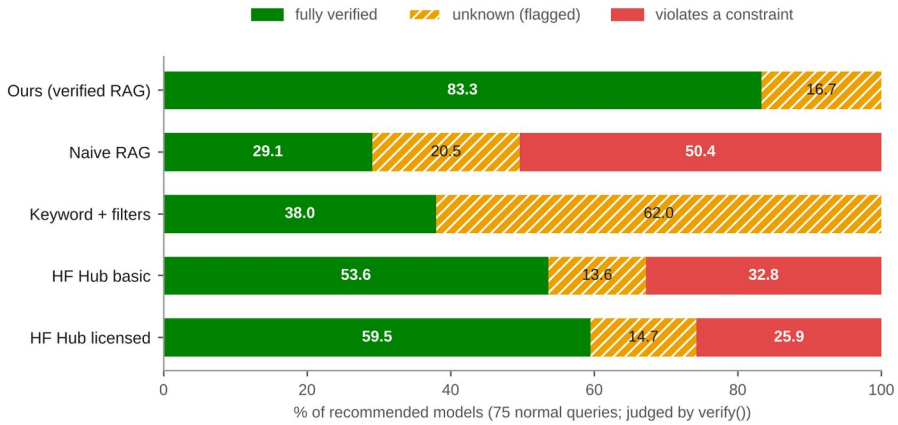


Fig. 4.3. How each system's suggestions were judged: fully checked, unknown, or breaking a rule.

Our tool suggested no model that breaks a rule, and 83.3% of its suggestions were fully checked, against 29.1% for plain RAG. It answered 73 of 75 questions; on the other two it asked a follow-up question. Plain RAG got the size wrong in 50.4% of its suggestions. The Hugging Face filters cannot filter by size at all and got it wrong in 80.0% (basic) and 71.9% (licence) of suggestions. Keyword search never

breaks a rule, because it uses our filters, but 62% of its suggestions are unknown: matching words says nothing about size or licence.

On the 15 vague questions our tool asked a follow-up question every time; the others just answered. Ten questions were meant to be impossible, but a model with unknown size or language never fails, so none could be proved impossible. On the 12 questions where no model in the data fully passes, our tool showed a banner every time and never claimed a model was fully checked; plain RAG and the Hugging Face filters suggested rule-breaking models for 63% to 75% of their suggestions.

4.3 Other checks

AI judges graded how well suggestions fit the request (Table 4.2). Our tool has the best mean grade, so checking the rules did not hurt relevance. A second pass by other judges agreed on 92.1% of grades (Cohen's kappa [12] 0.874, which is "almost perfect" agreement). This shows the judging is consistent, not that it matches people.

Table 4.2. Relevance (graded 0 to 2 by AI judges, blind).

System	Questions	Models graded	Mean grade	Questions with a grade-2 model
Ours	38	190	**1.28**	**81.6%**
Keyword + filters	40	197	1.06	65.0%
Plain RAG	40	200	0.87	67.5%
Hugging Face filters, licence	40	200	0.69	22.5%

AI judges also checked 98 claims in 28 explanations: 80 (81.6%) were backed by the model page, and half of the explanations were fully backed. So explanations are a convenience and not proof; the pass or fail tables do not depend on them. The starter code snippet ran on 20 of 32 small models (the rest need other tools). A normal search takes about 1.3 seconds, and 7 to 17 seconds with the explanation.

4.4 Limits

About 27% to 30% of models have no known size, and about 20% have no usable licence, so no method can check those; the 16.7% "unknown" is close to the best possible on this data. Grading was done by AI judges, not people. The questions were written by the same team that built the tool. The 100 questions cover the original five tasks only; the six newer tasks were spot-checked. Rights come from the *declared* licence and are not legal advice. The data is a snapshot of Hugging Face from 25 October 2026.

Chapter 5 Source Code

The code is at <https://github.com/Adityarane012/RAG-Project>. The part that decides every pass, fail or unknown is small and has no network access; it is shown below from `src/verify.py`.

Listing 5.1. src/verify.py (lines 73 to 90): one result for each rule the user gave.

```
def verify(meta: dict, constraints: dict) -> dict[str, str]:
    """Per-constraint result for one model, for the constraints that were requested."""
    results: dict[str, str] = {}
    if constraints.get("task"):
        results["task"] = check_task(meta, constraints["task"])
    if constraints.get("langs"):
        results["languages"] = check_languages(meta, constraints["langs"])
    if constraints.get("direction"):
        results["direction"] = check_direction(meta, constraints["direction"])
    size_limit = resolve_max_size_mb(constraints.get("max_size_mb"), constraints.get("size_word"))
    if size_limit is not None:
        results["size"] = check_size(meta, size_limit)
    if constraints.get("commercial"):
        results["commercial"] = check_right(meta, "commercial")
    if constraints.get("modifiable"):
        results["modifiable"] = check_right(meta, "modifiable")
    return results
```

Listing 5.2. src/verify.py (lines 106 to 123): combining the results and sorting.

```
def overall(results: dict[str, str]) -> str:
    """fail if anything fails, else unknown if anything is unknown, else pass."""
    values = results.values()
    if FAIL in values:
        return FAIL
    if UNKNOWN in values:
        return UNKNOWN
    return PASS


_RANK = {PASS: 0, UNKNOWN: 1, FAIL: 2}


def rank_key(results: dict[str, str]) -> tuple[int, int]:
    """Sort key: full passes first, then fewest unknowns (D12). Python's sort is
    stable, so the reranker's order is kept within each group."""
    n_unknown = sum(1 for v in results.values() if v == UNKNOWN)
    return (_RANK[overall(results)], n_unknown)
```

Chapter 6 Conclusion and Future Work

We built and tested HF Model Finder, which suggests Hugging Face models and checks each suggestion against the user's rules in code. Our tests support four conclusions. (1) Checking in code removed rule-breaking suggestions: none on 75 normal questions, against 50.4% for plain RAG and 25.9% to 32.8% for the Hugging Face filters. (2) Saying "unknown" is better than guessing: 83.3% of suggestions were fully checked, and the rest were clearly marked. (3) Checking many models and showing few helps (72.3% to 83.3% fully checked). (4) Checking did not hurt relevance: the AI judges gave our tool the best mean grade (1.28 out of 2).

Future work. (1) Have people grade a sample, to confirm the AI judges. (2) Write test questions for all eleven tasks. (3) Reduce "unknown" without guessing, for example by reading custom licence text and more file types. (4) Keep the data fresh automatically. (5) Run a user study comparing our tool with the Hugging Face search.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, Dec. 2023.

[3] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, Dec. 2009.

[5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, Jan. 2019.

[6] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, Feb. 2024.

[7] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. arXiv:2303.17580.

[8] W. Jiang *et al.*, "An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023, pp. 2463–2475.

[9] S. Longpre *et al.*, "A large-scale audit of dataset licensing and attribution in AI," *Nat. Mach. Intell.*, vol. 6, no. 8, pp. 975–987, Aug. 2024.

[10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023, pp. 79–90.

[11] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023. arXiv:2306.05685.

[12] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, Apr. 1960.

Appendix A: UML Activity Diagrams

Fig. A.1 shows the whole path of one question. Fig. A.2 shows how one model is checked, and Fig. A.3 shows the size rule. The diagrams use standard UML activity-diagram notation and were drawn with PlantUML from the text files in `report/source/uml`.

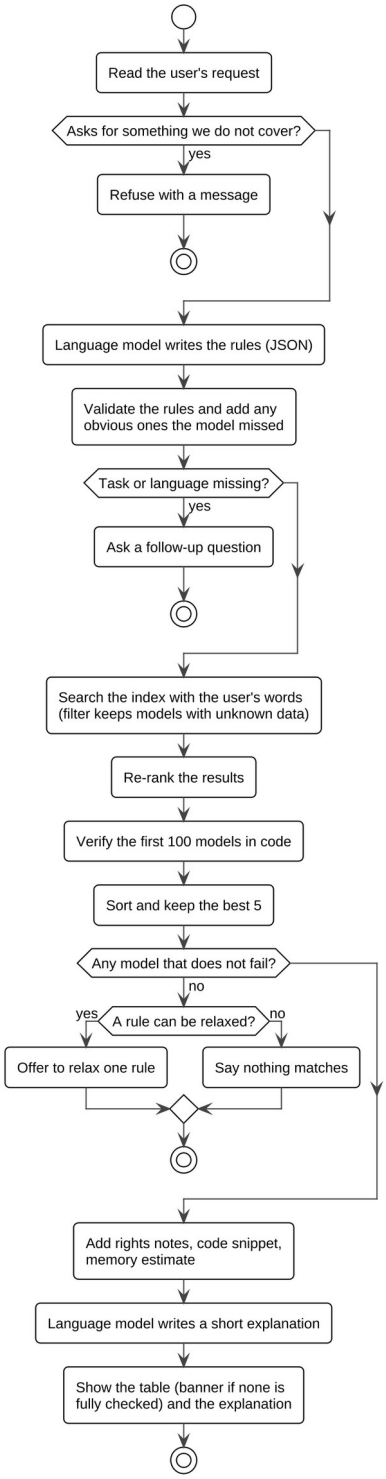


Fig. A.1. UML activity diagram of the question pipeline.

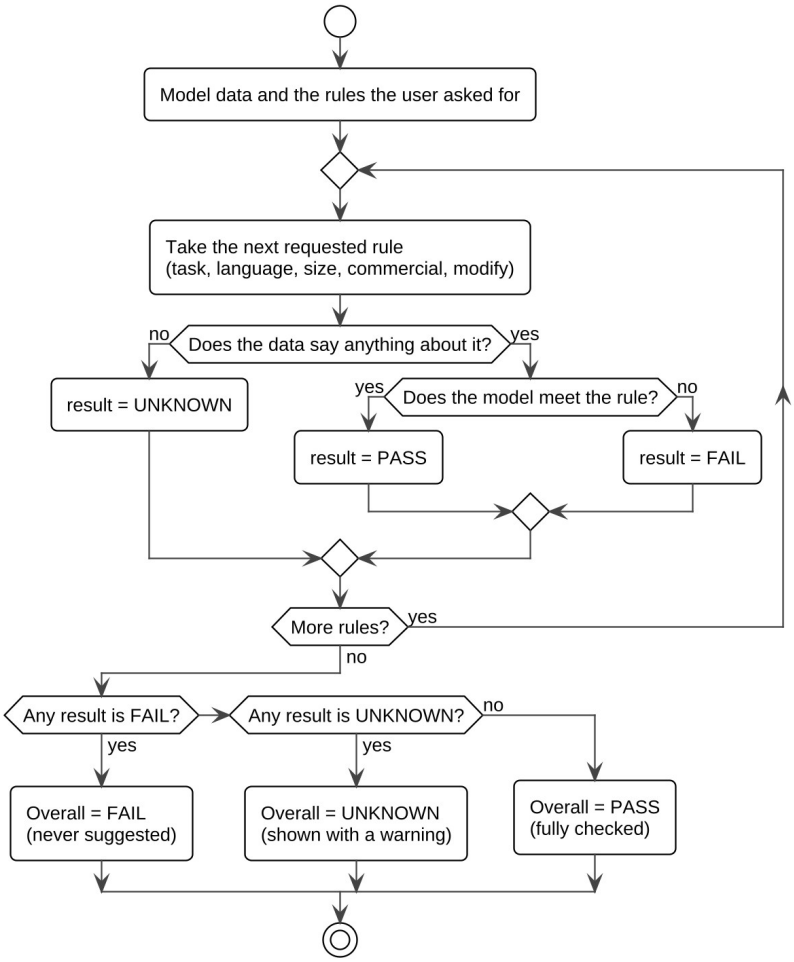


Fig. A.2. UML activity diagram of the check for one model: one result for each requested rule, then the combined result.

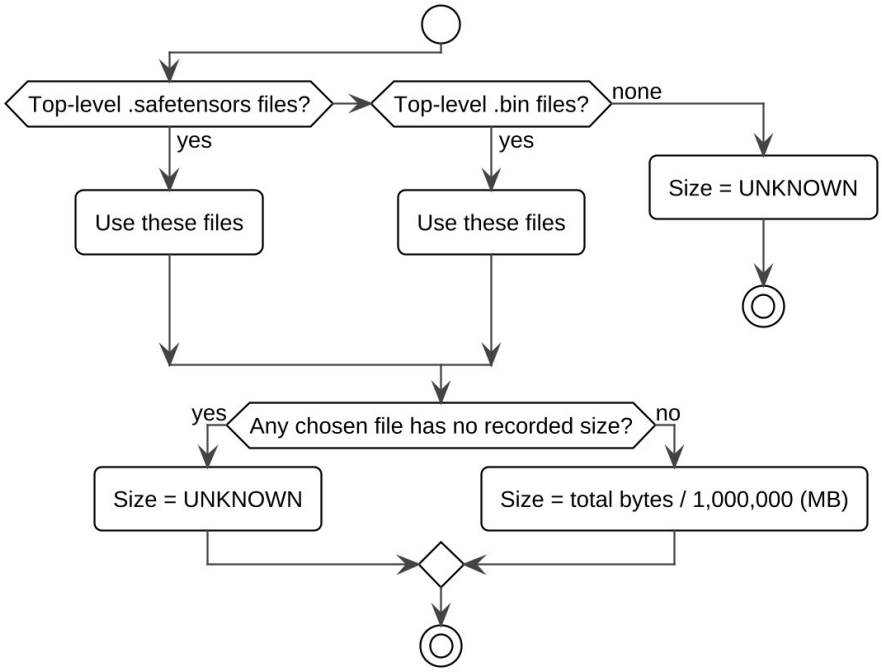


Fig. A.3. UML activity diagram of the size rule: the exact total of the main weight files, unknown if there is any doubt.